YAML-Grundlagen: Workflow-Dateien lesen und ändern

In YAML ist Einrückung Syntax, keine Dekoration. Ein Leerzeichen zu wenig, ein vergessener Bindestrich, und GitHub Actions liest die Workflow-Datei anders als gedacht oder gar nicht. In dieser Lektion lernst du, wie Schlüssel, Einrückung, Listen und mehrzeilige Werte aufgebaut sind. Danach änderst du einen Workflow-Schritt und prüfst die Änderung in einem manuellen Lauf.

> Lernmaterial: CC BY 4.0 (siehe `LICENSE-CONTENT.txt`).

Lernziel

Du kannst die Grundstruktur einer YAML-Workflow-Datei lesen, einen vorhandenen Schritt ändern und typische Einrückungs- und Wertfehler erkennen.

Was YAML in diesem Kurs macht

YAML ist eine Sprache zur Darstellung strukturierter Daten. Sie kann Schlüssel-Wert-Paare, Listen und Verschachtelungen beschreiben. Viele YAML-Dateien nutzen Einrückung, um die Struktur sichtbar zu machen; für Listen und Zuordnungen gibt es außerdem kompakte Schreibweisen mit Klammern.

GitHub Actions liest Workflows aus YAML-Dateien (`.yml` oder `.yaml`) im Verzeichnis `.github/workflows/`. Die Dateien teilen GitHub mit, wann ein Workflow startet und welche Jobs und Schritte er ausführt. Der folgende Ausschnitt zeigt den Anfang des Workflows aus Teil 1 (`.github/workflows/01-hallo-linux.yml`):

```yaml
name: 01 - Hallo Linux

on:
  workflow_dispatch:

jobs:
  erkunden:
    runs-on: ubuntu-latest
    steps:
      - name: Begrüßung
        run: echo "Hallo aus dem Linux-Runner!"
```

```
- name: Wer bin ich?
  run: whoami
```

Zwei Ebenen sind hier zu trennen:

- **YAML** legt die Struktur fest: Schlüssel, Werte, Listen, Einrückung.
- **GitHub Actions** legt fest, welche Schlüssel (`on`, `jobs`, `steps`, `runs-on`, `run` …) gültig sind und was sie bedeuten.

Der Wert von `run` ist ein Shell-Befehl, den der Runner ausführt. Welche Shell verwendet wird, hängt vom Betriebssystem des Runners ab; für Linux ist standardmäßig `bash` vorgesehen.[1]

GitHub dokumentiert die Workflow-Schlüssel und ihre Bedeutung in der Workflow-Syntax.

Schlüssel und Werte

Ein **Schlüssel** benennt eine Einstellung, der **Wert** gibt an, was dort eingetragen ist. Ein Doppelpunkt mit folgendem Leerzeichen trennt beide:

```
name: 01 - Hallo Linux
runs-on: ubuntu-latest
```

Das Leerzeichen nach dem Doppelpunkt ist Pflicht. `name:01` wäre kein Schlüssel-Wert-Paar, sondern ein einzelner Textwert.

Einrückung zeigt Zugehörigkeit

Die Einrückung legt fest, was zu wem gehört. Dafür gelten drei Regeln:

- Eingerückt wird ausschließlich mit **Leerzeichen**. Tabulatoren sind in YAML als Einrückung nicht erlaubt.
- **Gleichrangige** Einträge stehen gleich weit eingerückt, **untergeordnete** weiter rechts.
- Wie viele Leerzeichen eine Ebene hat, ist Konvention. In diesem Kurs sind es zwei pro Ebene.

[1] GitHub Actions Workflow-Syntax und YAML-Spezifikation 1.2.2, geprüft am 7. November 2026: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[2] und https://yaml.org/spec/1.2.2/[3]

```
jobs:
  erkunden:
    runs-on: ubuntu-latest
    steps:
      - name: Begrüßung
        run: echo "Hallo aus dem Linux-Runner!"
```

`erkunden` gehört zu `jobs`. `runs-on` und `steps` gehören zum Job `erkunden`. `name` und `run` gehören zum ersten Schritt. Die Hierarchie lautet also: `jobs → erkunden → steps → Schritt → name/run`.

Um mehrere Jobs nebeneinander zu sehen, hilft ein Beispiel mit zwei Jobs:

```
jobs:
  vorbereiten:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Vorbereitung"
  pruefen:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Prüfung"
```

`vorbereiten` und `pruefen` stehen auf derselben Ebene unter `jobs`. Beide haben dieselbe Struktur darunter.

Aufgabe: Öffne `.github/workflows/05-mehrere-jobs.yml` in deinem Lern-Repository und verfolge die Einrückung von `jobs` bis zu den einzelnen Schritten.

Kontrolle: Unter `jobs` stehen mehrere Jobs auf einer Ebene, jeder mit eigenem `steps`. Das Grundmuster entspricht `01-hallo-linux.yml`, nur mit mehr Jobs.

Listen

Eine **Liste** hat mehrere Einträge, jeder beginnt mit einem Bindestrich und einem Leerzeichen. Die Schritte eines Jobs sind eine Liste:

```
steps:
  - name: Begrüßung
    run: echo "Hallo aus dem Linux-Runner!"
  - name: Wer bin ich?
    run: whoami
```

Der Bindestrich steht für den Beginn eines neuen Eintrags. Die Schlüssel danach (`name`, `run`) gehören zu diesem Eintrag und müssen untereinander auf gleicher Höhe stehen. Das ist die häufigste Fehlerquelle bei `steps`: Rückt `run` weiter ein oder weniger weit als `name`, ist die Struktur kaputt oder bedeutet etwas anderes.

Aufgabe: Zähle im Beispiel die Schritte und benenne sie.

Kontrolle: Es sind zwei Schritte: „Begrüßung“ und „Wer bin ich?“.

Werte, die YAML eigenmächtig deutet

YAML kann Werte als Text, Zahl, Wahrheitswert oder „kein Wert“ interpretieren. Welche Typen erkannt werden, hängt auch vom verwendeten Schema und Parser ab. Viele YAML-1.2-Parser mit Core-Schema interpretieren `true` und `false` als Wahrheitswerte, `42` als Zahl und `null` sowie `~` als „kein Wert“. Sie lesen `3.10` als Zahl 3,1.[4]

Für Versionsangaben und ähnliche Werte ist das tückisch. Schreibe Versionsangaben deshalb als Text in Anführungszeichen:

```
python-version: "3.10"
```

Anführungszeichen sind außerdem nötig, wenn ein Text sonst falsch gelesen würde:

- Enthält ein einfacher Textwert Doppelpunkt plus Leerzeichen (`:`) oder Leerzeichen plus Raute (`#`), kann YAML ihn als Struktur oder Kommentar lesen. Anführungszeichen machen deutlich, dass der ganze Wert Text ist.
- Ein Cron-Ausdruck wie `* * * * *` beginnt mit einem YAML-Sonderzeichen. Setze solche Werte in Anführungszeichen.

[4] GitHub Actions Workflow-Syntax und YAML-Spezifikation 1.2.2, geprüft am 7. November 2026: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[5] und https://yaml.org/spec/1.2.2/[6]

Einfache Anführungszeichen (`'…'`) nehmen den Text wörtlich. In doppelten Anführungszeichen (`"…"`) gelten Escape-Sequenzen wie `\n`.

Ändere vorhandene Werte mit Anführungszeichen nicht ohne Grund: Sie stehen meist dort, weil der Wert sonst falsch interpretiert würde.

Mehrzeilige Werte mit `run: |`

Ein senkrechter Strich (`|`) leitet einen mehrzeiligen Textblock ein und erhält die Zeilenumbrüche. Die Shell interpretiert anschließend den Text; in den folgenden Beispielen steht jeweils ein Befehl pro Zeile.

```
jobs:
  dateien:
    runs-on: ubuntu-latest
    steps:
      - name: Ordner und Dateien anlegen
        run: |
          mkdir -p arbeit
          touch arbeit/notizen.txt
          echo "Erste Notiz" > arbeit/notizen.txt
          ls -l arbeit
```

Die eingerückten Zeilen unter `run: |` sind keine YAML-Schlüssel. YAML grenzt nur den Textblock ab, die Shell verarbeitet ihn anschließend. Der Block endet, sobald eine Zeile weniger eingerückt ist als sein erster Inhalt. Der Ausschnitt entspricht dem Schritt in `.github/workflows/02-dateien.yml`; dort steht der Job zusätzlich mit `runs-on: ubuntu-latest`.

Daneben gibt es `>`. Dort werden Zeilenumbrüche zu Leerzeichen zusammengefasst. Für mehrzeilige Shell-Befehle ist `|` die passende Wahl.

Aufgabe: Im gezeigten Block ist `touch arbeit/notizen.txt` korrekt als Shell-Befehl eingerückt. Vergleiche seine Einrückung mit der Zeile `mkdir -p arbeit`. Was würde sich ändern, wenn du `touch` ganz links bündig schreibst?

Kontrolle: Ganz links beginnt die Zeile außerhalb des `run`-Textblocks. Sie ist dann kein Befehl dieses Schritts mehr; ob der Workflow noch gültig ist, hängt davon ab, wie YAML die übrige Datei danach einordnet. Ändere die Einrückung im echten Workflow nicht.

Kommentare

Ein `#` beginnt einen Kommentar, wenn es am Zeilenanfang steht oder davor ein Leerzeichen steht. Der Kommentar reicht bis zum Zeilenende. In `farbe: a#b` ist `#` dagegen Teil des Werts. Kommentare sind Hinweise für Menschen und haben keine Wirkung auf den Workflow.

Syntax prüfen und Änderungen kontrollieren

Syntaxfehler (falsche Einrückung, fehlender Doppelpunkt, falsch platzierter Bindestrich) und inhaltliche Fehler (unbekannter Schlüssel, falscher Befehl) sind zweierlei:

- Bei Syntaxfehlern kann GitHub die Workflow-Datei nicht auswerten; der Workflow startet nicht. Die genaue Meldung kann sich je nach Oberfläche ändern.
- Ein inhaltlicher Fehler kann erst beim Lauf sichtbar werden: Der Workflow startet, aber ein Schritt schlägt fehl.

Übung: Ändere nur den `name:`-Wert eines vorhandenen Schritts in `.github/workflows/01-hallo-linux.yml`.

1. Bearbeite die Datei in deinem Lern-Repository und achte auf die Einrückung.
2. Vergleiche den geänderten Abschnitt mit einem unveränderten Beispiel.
3. Committe die Änderung auf einem Arbeitsbranch (Ablauf siehe Git-Grundlagen).
4. Öffne **Actions**, wähle **01 - Hallo Linux**, klicke **Run workflow**, wähle deinen Arbeitsbranch und starte den Lauf.

Kontrolle: Prüfe drei Dinge getrennt:

- GitHub zeigt den Workflow an.
- Der Lauf startet.
- Der Schritt trägt im Protokoll den neuen Namen.

Dass die Datei angezeigt wird, belegt noch nicht, dass der Workflow erfolgreich läuft. Für den manuellen Start gelten dieselben Regeln wie in der Git-Lektion: `workflow_dispatch` muss in der Datei stehen, und die Datei muss auf dem Standardbranch liegen.

Typische Fehler

- **Schritt fehlt oder liegt im falschen Job:** Der Bindestrich steht an der falschen Stelle oder auf der falschen Ebene unter `steps`.
- **Schlüssel innerhalb eines Schritts rutscht heraus:** `name` und `run` stehen nicht auf gleicher Höhe.
- **Tabulator statt Leerzeichen:** Viele Editoren fügen Tabulatoren ein. YAML lehnt sie als Einrückung ab.
- **Version wird als Zahl gelesen:** Viele YAML-1.2-Parser mit Core-Schema lesen `3.10` ohne Anführungszeichen als `3.1`.
- **Wert mit `:` oder `#` zerfällt:** Der Wert gehört in Anführungszeichen.
- **Shell-Zeile wird als YAML gelesen:** Die Zeile ist weniger eingerückt als der Rest des `run: |`-Blocks.

Weiterlernen

GitHub dokumentiert die unterstützten Schlüssel und Regeln der Workflow-Dateien. Die YAML-Spezifikation beschreibt die Sprache und ihre Datentypen; sie trägt die Versionsangabe 1.2.2 (Revision vom 1. Oktober 2021).[7] GitHub verwendet für den Runner `ubuntu-latest` standardmäßig Bash für `run`-Schritte.[10]

Glossar

- **Einrückung:** Leerzeichen am Zeilenanfang, die in YAML die Zugehörigkeit zeigen.
- **Job:** Zusammengehörige Schritte, die auf einem Runner laufen.
- **Liste:** Folge von Einträgen, jeder mit `-` eingeleitet.
- **Schlüssel (key):** Name einer Einstellung.
- **Shell:** Programm, das Befehle entgegennimmt und ausführt.
- **Textblock:** Mehrzeiliger Textwert, eingeleitet mit `|` oder `>`.
- **Wert (value):** Inhalt, der einem Schlüssel zugeordnet ist.
- **Workflow:** Automatisierter Ablauf in GitHub Actions aus Jobs und Schritten.
- **Workflow-Datei:** YAML-Datei in `.github/workflows/` mit der Konfiguration eines Workflows.

[7] GitHub Actions Workflow-Syntax und YAML-Spezifikation 1.2.2, geprüft am 7. November 2026: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax[8] und https://yaml.org/spec/1.2.2/[9]

[10] GitHub Actions Workflow-Syntax, Abschnitt „Default shell and working directory“, geprüft am 7. November 2026: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#default-shell-and-working-directory[11]

- **YAML:** Textformat für strukturierte Daten, das Struktur über Einrückung abbildet.

Begriffe wie Trigger, `workflow_dispatch` und GitHub Actions stehen im Glossar der Git-Lektion.

Schlagworte

`YAML` `GitHub Actions` `Workflow` `Workflow-Datei` `Einrückung` `Shell` `Konfiguration`